

Visual Positioning

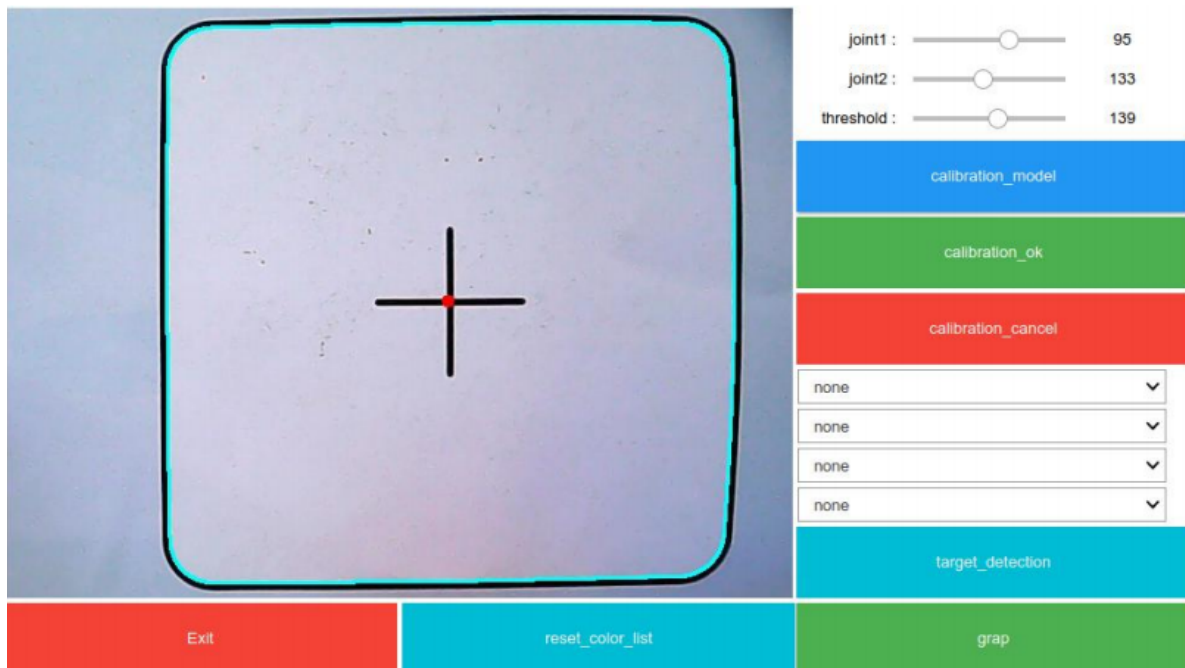
This section contains theoretical knowledge. It is not a standalone gameplay mode, but a prerequisite and theoretical foundation for learning courses [7. Color Sorting and Stacking] and [8. Garbage Sorting].

```
#Run the gameplay
dofbot_ws/src/dofbot_color_identify/scripts/颜色分拣color_sort.ipynb
#Or
dofbot_ws/src/dofbot_color_stacking/scripts/颜色堆叠color_stacking_play.ipynb
#You can see the following example images
```

Frame Calibration

The purpose of calibration is to convert the position of the object's center in the field of view to the actual distance from the robotic arm's base coordinate position.

- Example image



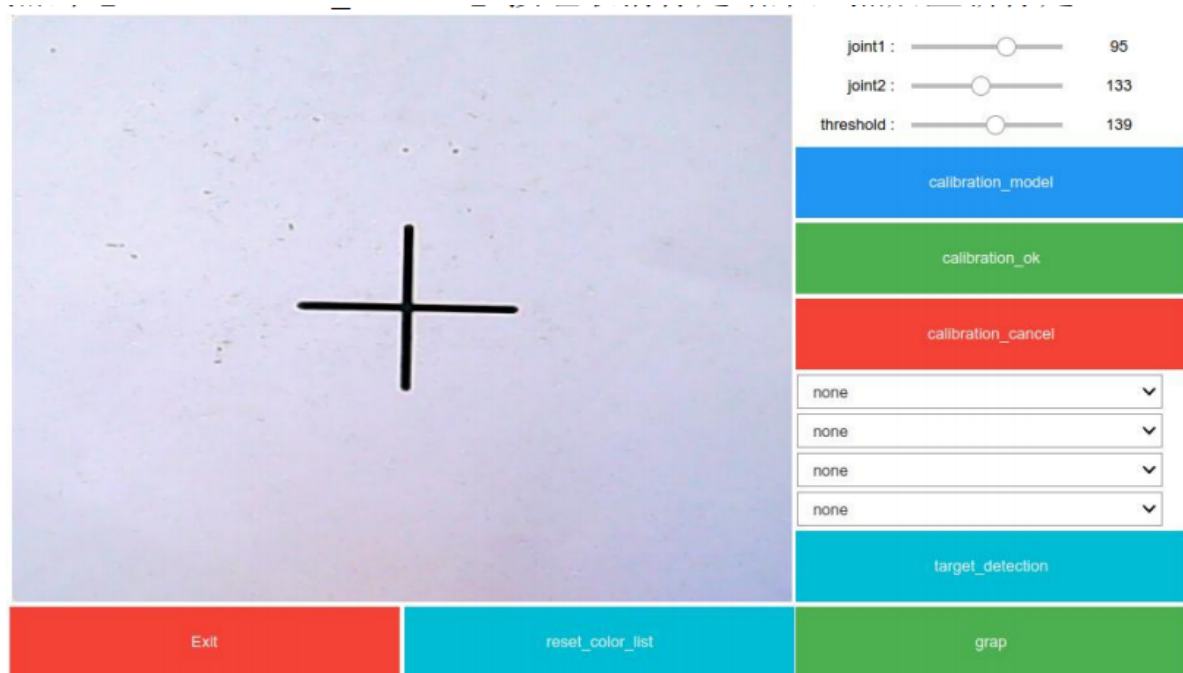
- Operation process

(1) After starting all code blocks, the interface is displayed at the bottom of the code. Click the [calibration_model] button to enter calibration mode. Then you can adjust the slider.

(2) Adjust the slider so that the frame completely appears in the field of view and the edges of the frame are fully detected. Slider [joint1] controls servo 1 for left-right adjustment, slider [joint2] controls servo 2 for up-down adjustment, slider [threshold] adjusts the image edge binarization detection threshold. Note: The four edges of the frame must be clearly identified without external interference.

(3) Click the [calibration_ok] button to complete calibration. At the same time, the adjusted parameters will be written to the configuration file. When starting next time, if there are no issues, no adjustment is needed - just click calibrate and confirm.

(4) At this point, calibration is complete, as shown in the figure below. If an error occurs during calibration, click the [calibration_cancel] button to cancel the calibration result, then recalibrate.



- Code design

Image processing, extracting contour points

```
# Convert image to grayscale
gray = cv.cvtColor(self.image, cv.COLOR_BGR2GRAY)
# Use Gaussian blur on the image
gray = cv.GaussianBlur(gray, (5, 5), 1)
# Image binarization operation
ref, threshold = cv.threshold(gray, self.threshold_num, 255,
cv.THRESH_BINARY)
# Get structuring elements of different shapes
kernel = np.ones((3, 3), np.uint8)
# Morphological opening operation
blur = cv.morphologyEx(threshold, cv.MORPH_OPEN, kernel, iterations=4)
# Extraction mode
mode = cv.RETR_EXTERNAL
# Extraction method
method = cv.CHAIN_APPROX_NONE
# Get contour point set (coordinates) - slight difference between python2 and
python3
# Hierarchical relationship
# Parameter 1: Input binary image, Parameter 2: extraction mode, Parameter 3:
extraction method
contours, hierarchy = cv.findContours(blur, mode, method)
```

Draw contours, return edge points of contours

```
# Traverse point set
for i, c in enumerate(contours):
# Calculate contour area
area = cv.contourArea(c)
# Set contour area range
if h * w / 2 < area < h * w:
# Calculate moments of polygon
```

```

mm = cv.moments(c)
cx = mm['m10'] / mm['m00']
cy = mm['m01'] / mm['m00']
# Draw contour area
cv.drawContours(self.image, contours, i, (255, 255, 0), 2)
# Get contour edge points
dp = np.squeeze(cv.approxPolyDP(c, 100, True))
# Draw center
cv.circle(self.image, (int(cx), int(cy)), 5, (0, 0, 255), -1)

```

Perspective transformation, transform the detected contour into a (640,480) image

```

#Classify the contour points detected by edge detection into top-left, bottom-
left, bottom-right, top-right
for i in range(len(dp)):
if dp[i][0]<320 and dp[i][1]<240: upper_left=dp[i]
if dp[i][0]<320 and dp[i][1]>240: lower_left=dp[i]
if dp[i][0]>320 and dp[i][1]>240: lower_right=dp[i]
if dp[i][0]>320 and dp[i][1]<240: upper_right=dp[i]
# The four vertices in the original image
pts1 = np.float32([upper_left, lower_left, lower_right, upper_right])
# The four vertices after transformation
pts2 = np.float32([[0, 0], [0, 480], [640, 480], [640, 0]])
# Calculate perspective transformation from four pairs of corresponding points
M = cv.getPerspectiveTransform(pts1, pts2)
# Apply perspective transformation to the image
Transform_img = cv.warpPerspective(image, M, (640, 480))

```

For detailed code, see

dofbot_ws/src/dofbot_color_identify/scripts/dofbot_config.py

Or

dofbot_ws/src/dofbot_color_stacking/scripts/dofbot_config.py

Positioning Calculation

- Object positioning calculation in the field of view

The camera on the robotic arm sees the widest field of view of the ground at a 45-degree oblique angle. At this angle, the frame in the field of view appears trapezoidal. After perspective transformation, the four edges of the frame are transformed into the four edges of the image.

Therefore, there is a proportional relationship between the distance from the object's center point in the field of view to the center of the image (640,480) and the actual distance from the object's center to the center of the frame (160,160).

Lateral calculation, longitudinal calculation is similar:

$$(point_x - image_x) / 640 = (object_x - x) / 160$$

Based on the robotic arm base coordinate system (introduced in URDF visualization)

point_x: Lateral coordinate of the object's center point in the field of view

image_x: Lateral coordinate of the image center point

object_x: Lateral coordinate of the actual object's center point

x: Position of the base coordinate system, which is 0 at this time

160: Frame side length in millimeters

In random sorting, only object_x is unknown and random; all others are known

- Positioning calculation example code

```
(a, b) = (round(((point_x - 320) / 4000), 5), round(((480 - point_y) /  
3000) * 0.8+0.19, 5))
```

(a,b): Coordinates in the base coordinate system, converted from the object's center point coordinates in the field of view.

(point_x - 320) / 4000: Distance of the object from the frame center point in the x direction.

(480- point_y) / 3000: Distance of the object from the frame center point in the y direction.

0.19: Coordinate of the base coordinate center point to the nearest edge of the frame, manually measured, there is some deviation, can be adjusted appropriately.

0.8: Adjustment for errors caused by different distances from the camera, can be adjusted appropriately.

((480 - point_y) / 3000) * 0.8+0.19: Distance of the object from the base coordinate center point in the y direction.

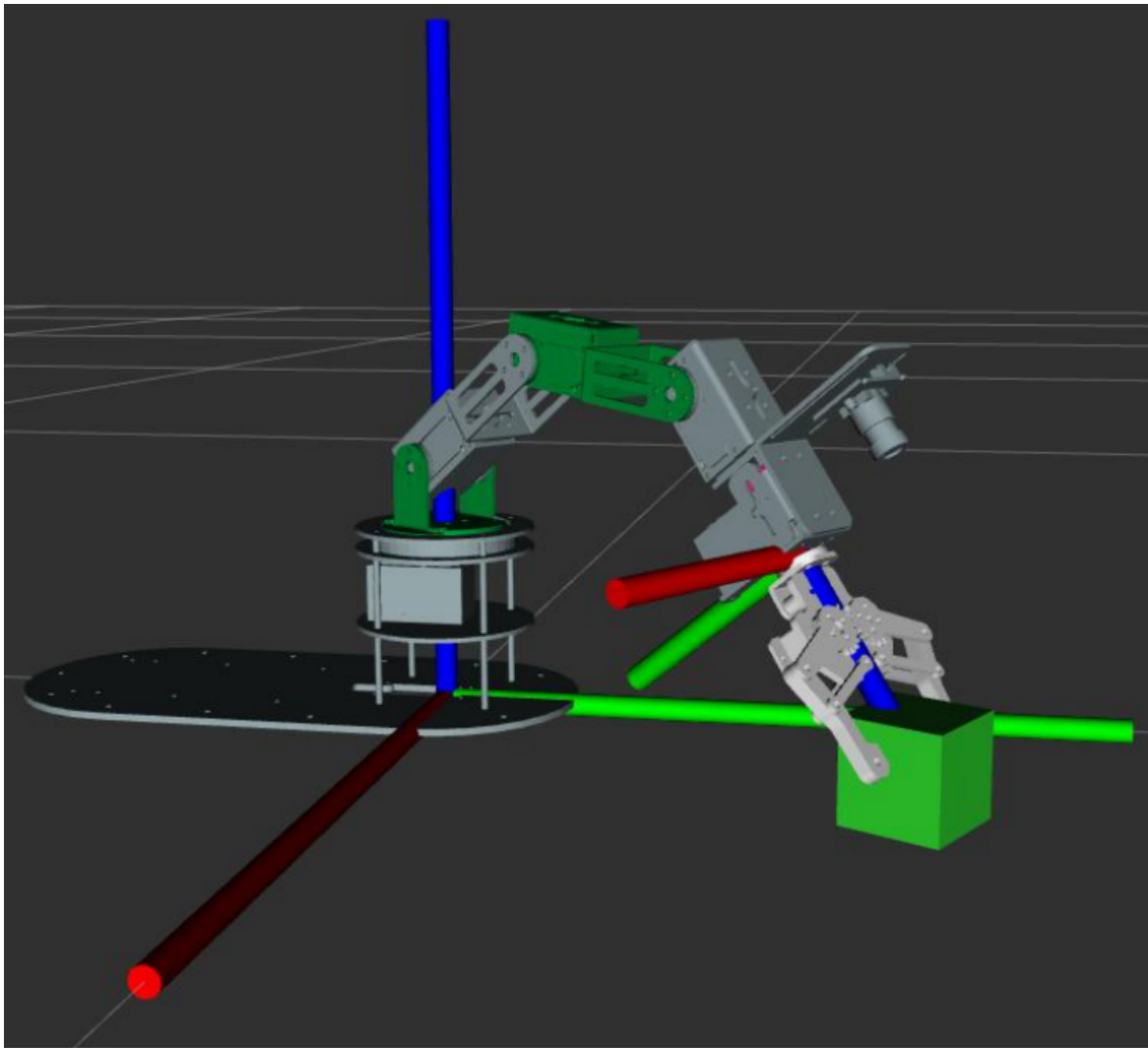
- Inverse kinematics angle adjustment

After obtaining the object's positioning in the base coordinate system, inverse kinematics calculates the angles each joint of the robotic arm needs to rotate to reach the specified posture.

Since the movement range of the robotic arm joints is 0°~180°, negative angle values cannot be reached. When the inverse solution exceeds the limit and a negative value appears, adjust appropriately.

```
if joints[2] < 0:  
    joints[1] += joints[2] * 3 / 5  
    joints[3] += joints[2] * 3 / 5  
    joints[2] = 0
```

Gripper Calculation



Since different end effectors are generally used for different use scenarios, the inverse kinematics solution calculates the pose of the robotic arm's end, not the end of the end effector (gripper, suction cup, etc.).

The chassis coordinate system is called the **base coordinate system**, the red-green-blue coordinate system at the gripper is called the **end coordinate system**, and the gripper is called the **end effector**. Now, the inverse kinematics solution calculates the position the **end coordinate system** (not the gripper's grasping position) needs to reach. For the robotic arm shown in the figure, before solving the inverse, we need to obtain the position information the end coordinate system needs to reach (i.e., the pose information of the end coordinate system). So how to solve this?

For the pose, we set clockwise rotation around the axis (negative for counterclockwise, positive for clockwise) as follows:

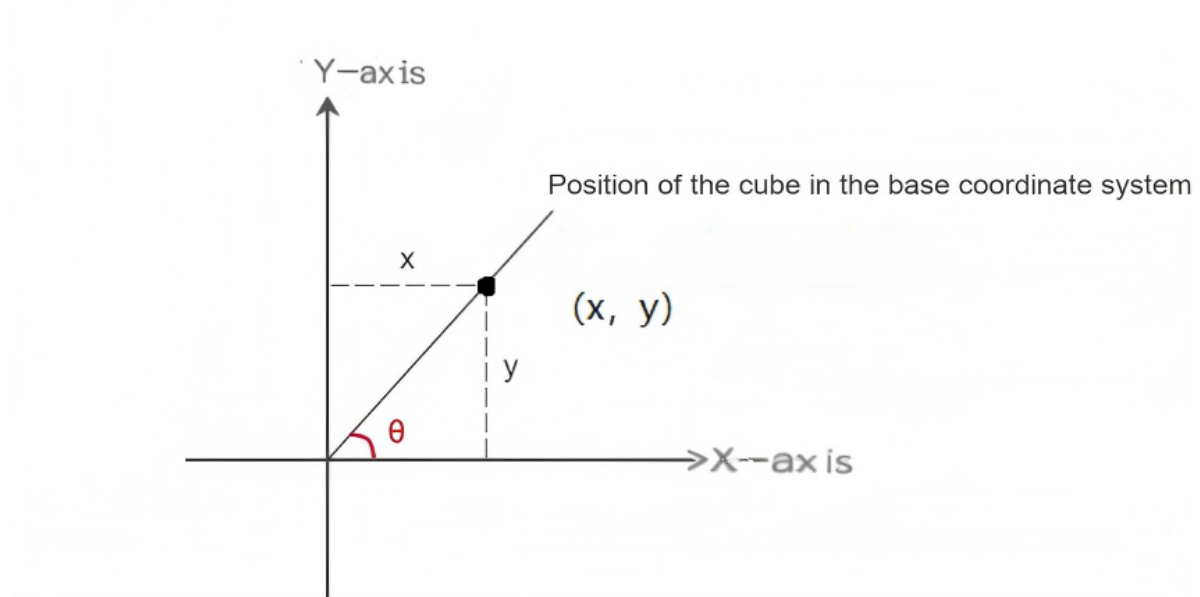
```
// Set different grasping poses according to object distance

double Roll = 2.5 * request.tar_y * 100 - 207.5;

double Pitch = 0;

double Yaw = 0;
```

- Calculate offset angle



```
double init_angle = atan2(double(request.tar_y), double(request.tar_x));
```

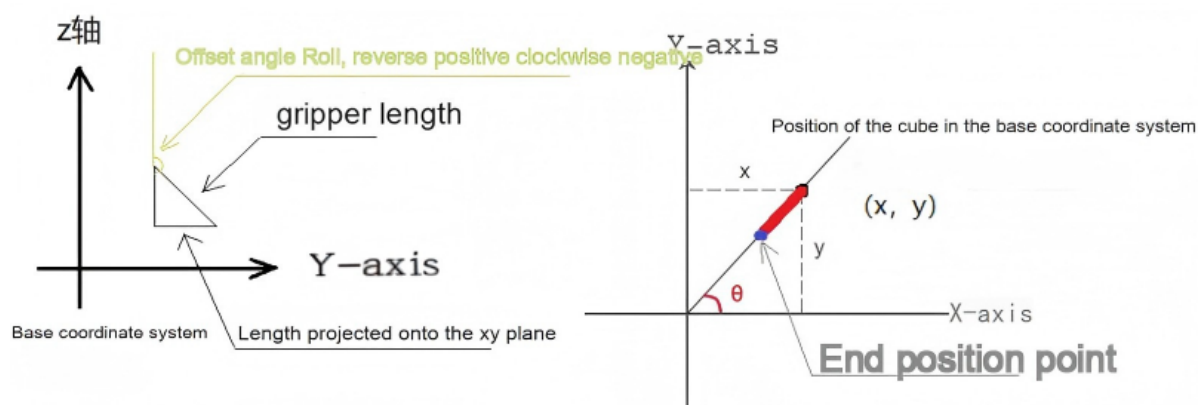
atan2() is the arctangent function, an enhanced version of atan() that can determine the quadrant double

atan2(double y, double x);

The function of atan2() is to calculate the arctangent value of y/x . atan2() is an enhanced version of atan() that can determine the quadrant. The arctangent function atan2() and the tangent function tan() have opposite functions:

tan() is to find the tangent value of an angle given the radian value of the angle; while atan2() is to find the radian value of an angle given the tangent value (i.e., y/x).

- Find the projection length of the gripper on the hypotenuse, corresponding to the xy plane, as shown in the right figure



Measure the gripper length, tool_param=0.12; unit is meters.

```
double dist = tool_param * sin((180 + Roll) * DE2RA);
```

- Find the length of the robotic arm projected onto the xy plane (excluding the gripper part), the corresponding distance is shown in the upper right figure, excluding the red part.

```
double distance = hypot(request.tar_x, request.tar_y) - dist;
```

- Find the end position (excluding gripper), the corresponding position is the x, y coordinates shown in the upper right figure

```
double x = distance * cos(init_angle);
```

```
double y = distance * sin(init_angle);
```

- The final step is to find the z-axis coordinate

We need to find the z coordinate of the end coordinate system in the base coordinate system. See the left upper figure triangle, use trigonometric functions to find the vertical leg height.

```
double z = tool_param * cos((180 + Roll) * DE2RA);
```